



Universidade do Minho
Escola de Engenharia

Algoritmos e Estruturas de Dados 2

Universidade do Minho

2024/2025

Relatório Final

PL82 – Dia 20 de Maio de 2025

Ana Íris Machado Torres (IT) – a110489

Francisca da Silva Fernandes de Sousa Carvalho (FC) – a112190

Mariana Silva Gomes (MG) – a110943

Matilde Pires Cunha (MC) – a111787

Índice:

1. Planeamento Semanal	3
2. Anotações do Estudo do Enunciado	5
3. Análise do problema.....	6
3.1. Funcionalidades do programa:	6
3.2. Requisitos	7
3.3. Histórias de utilização.....	8
4. Conceção do programa	9
5. Desenvolvimento	10
6. Anexos.....	16
6.1. Teste de funções:	16

1. Planeamento Semanal

Tabela 1 – Plano semanal

Semana	Tarefas	Recursos	Esforço previsto	Esforço realizado
1 (14/03)	Fazer o planeamento semanal Organização do grupo Submissão do relatório inicial	Todas Todas IT	3h 30 min 1 min	3h 30 min 1 min
2 (21/03)	Atualizar o planeamento semanal Melhoramento do relatório Estudar o enunciado	Todas Todas Todas	1h 1h 2h	1h 1h 2h
3 (28/03)	Atualizar o planeamento semanal Melhoramento do relatório final Levantamento dos requisitos Levantamento das histórias de utilização Definir a estrutura de dados Elaborar um protótipo das funções	MG e MC Todas FC FC e MG IT e MG MC e IT	1h 1h 30 min 2h 1h 30 min	1h 1h 30 min 2h 1h 30 min
4 (04/04)	Atualizar o planeamento semanal Melhoramento do relatório final Atualização dos requisitos Atualização das histórias de utilização Tomada de decisão sobre o formato do ficheiro do programa Implementar novas funcionalidades no programa • Função criaEncomenda • Função insereEncomendaPendente • Função enqueue • Função main Testar as funcionalidades	FC e IT MG Todas MC e IT MG Todas Todas	1h 1h 30 min 1h 10 min 3h 1h	1h 1h 30 min 1h 10 min 3h 1h
5 (16/04)	Atualizar o planeamento semanal Melhoramento do relatório final Implementação dos ficheiros • encomendasPendentes • encomendasExpedidas • histórico Implementar novas funcionalidades no programa • Função validaData	MG e FC IT e MC Todas Todas	1h 30 min 30 min 4h	1h 1h 30 min 5h

	• Função transferirPendentes • Função trasnferirExpedidas • Função transferir Historico • Função limpaEnter • Função capturaEntrada • Função insereEncomendaPendente • Função listarEncomendasPendentes • Função enqueue • Função front • Função push • Função Peek Testar as funcionalidades	Todas	1h	3h
6 (25/04)	Reajuste do código • Função separarElementos • Função validaData • Função Peek • Função idHistorico • Função idData • Função idNome Testar todas as funcionalidades Terminar o programa	FC e MC	2h30	4h
		Todas IT e MG	30 min 2h	1h 3h20

2. Anotações do Estudo do Enunciado

#define numEcomendas 200

- Vamos começar por fazer uma estrutura para cada encomenda:

```
typedef struct encomenda {  
    Int ID;  
    Char nome[100];  
    Char descrição[100];  
    Char dataDeCriacao[9]; // AAAAMMDD  
}  
Encomenda E[numEncomendas];
```

- **Lista ligada** para encomendas pendentes (inserir, remover e listar).
- **Fila** para a expedição de encomendas (mover da lista, enviar e listar).
- **Pilha** para o histórico de encomendas expedidas (armazenar e listar por ID, data ou cliente).

Encomendas pendentes (lista ligadas):

- 1º Vamos **adicionar** encomendas pendentes;
- 2º Vamos **listar** encomendas pendentes;
- 3º Vamos **remover** encomendas pendentes.

Encomendas para expedição (fila):

- 4º **Adicionar** na encomendar para expedição;
- 5º **Remover** a encomenda para expedição (foi enviado para o cliente);
- 6º **Listar** as encomendas na lista de expedição (as que sobraram).

Histórico (pilha):

- 7º A encomenda removida da expedição **acrescentamos** no histórico;
- 8º **listar** encomendas expedidas (ou seja, as que já foram despachadas).

Nota: Optámos por guardar os ficheiros à medida que realizávamos cada operação, em vez de apenas no final, para garantir que os dados ficavam sempre atualizados e seguros mesmo que o programa fosse interrompido inesperadamente.

3. Análise do problema

O projeto realizado tem como principal objetivo: o desenvolvimento de um sistema de gestão de encomendas para uma loja online, na linguagem C.

3.1. Funcionalidades do programa:

❖ **Entrada de dados:** Leitura de ficheiros de texto, que contêm os dados das encomendas pendentes, em expedição e no histórico.

❖ **Estruturas de dados:** Utilização de:

- Lista ligada para as encomendas pendentes;
- Fila para as encomendas em expedição;
- Pilha para o histórico de encomendas enviadas.

❖ **Incremento de encomendas:** Permite ao utilizador introduzir novas encomendas com o nome, descrição e data de criação e com a validação do formato.

❖ **Remoção de encomendas:** Remoção de encomendas pendentes e inserção na fila de expedição, mantendo desta forma, a ordem de chegada.

❖ **Envio de encomendas:** Encomendas em expedição são enviadas e transferidas para o histórico, com registo da ordem de envio.

❖ **Consulta do histórico:** Possibilidade de listar as encomendas enviadas por:

- Intervalo de IDs;
- Intervalo de datas;
- Nome do cliente (total ou parcial).

❖ **Validação dos dados:** Verificação rigorosa das datas, incluindo anos bissextos, e exclusividade de IDs em todas as estruturas.

❖ **Persistência de dados:** Atualização automática dos ficheiros após cada operação, garantindo que os dados são guardados entre cada execução.

❖ **Interface interativa:** Menu com linha de comandos com opções numeradas e mensagens claras para o utilizador.

3.2. Requisitos

O sistema de gestão de encomendas desenvolvido neste projeto, tem como base, um conjunto de requisitos funcionais que garantem a utilidade, organização e eficiência do mesmo.

- ❖ **Gestão de uma lista de encomendas;**
- ❖ **Gestão da lista de expedição de encomendas;**
- ❖ **Gestão do histórico de encomendas enviadas:**
 - Possibilidade de guardar e recuperar informação a partir de ficheiros

- ❖ **Guardar dados de cliente:**
 - ID único;
 - Nome do cliente;
 - Descrição dos produtos (encomenda);
 - Data de criação de encomenda;

- ❖ **Nas encomendas pendentes, é necessário se poder:**
 - Inserir, remover e listar todas as encomendas.

- ❖ **Nas encomendas expedidas:**
 - Adicionar uma encomenda para expedir:
 - Automaticamente removendo-a da lista de encomendas pendentes;
 - Listar as encomendas na fila de expedição.

- ❖ **Histórico de encomendas:**
 - Acrescentar uma encomenda expedida:
 - Removendo-a da lista de encomendas em expedição;
 - Listar todas as encomendas expedidas através do ID ou data de encomenda ou nome do cliente.

3.3. Histórias de utilização

Tabela 2 – Histórias de atualização

Tipo de Utilizador	História de Utilização	Funcionalidade relacionada
Funcionário	Como funcionário, quero inserir a encomenda do meu cliente.	Criar encomenda
Funcionário	Como funcionário, quero visualizar todas as encomendas pendentes, para acompanhar o estado das compras dos clientes.	Listar encomendas pendentes
Funcionário	Como funcionário, quero mover uma encomenda pendente para a fila de expedição para que seja processada para envio.	Adicionar encomenda à fila de expedição
Funcionário	Como funcionário, quero visualizar todas as encomendas de expedição, para acompanhar o estado das compras dos clientes.	Listar encomendas de expedição
Funcionário	Como funcionário, pretendo enviar uma encomenda da fila de expedição para o cliente receber a sua encomenda.	Enviar uma encomenda
Sistema	Como sistema, quero armazenar as encomendas enviadas no histórico para manter um registo das transações concluídas.	Adicionar encomenda ao histórico
Funcionário	Como funcionário, quero aceder ao histórico das encomendas, filtrando por ordem de ID, data de expedição e nome do cliente.	Consultar histórico de encomendas
Sistema	Como sistema, quero guardar e recuperar todas as encomendas em ficheiros para garantir a persistência dos dados.	Guardar e carregar dados

4. Conceção do programa

Bibliotecas:

- ❖ stdio.h;
- ❖ stdlib.h;
- ❖ string.h
- ❖ locale.h.

Esquema de estrutura de dados:

- i) Armazenar os dados das encomendas;
- ii) Declarar a lista ligada das encomendas pendentes;
- iii) Gerir a fila de encomendas em expedição;
- iv) Gerir a pilha do histórico de encomendas enviadas;

Lista de funções:

- a) Validar datas no formato AAAAMMDD;
- b) Verificar se já existe um ID em qualquer estrutura;
- c) Separar os campos de uma linha lida do ficheiro;
- d) Ler o ficheiro de encomendas pendentes e carregar para a lista ligada;
- e) Ler o ficheiro de encomendas em expedição e carregar para a fila;
- f) Ler o ficheiro do histórico e carregar para a pilha;
- g) Criar uma encomenda com um ID exclusivo;
- h) Inserir uma nova encomenda na lista de pendentes e atualizar o ficheiro;
- i) Listar todas as encomendas pendentes;
- j) Listar todas as encomendas na fila de expedição;
- k) Listar encomendas do histórico por nome do cliente, com filtros adicionais;
- l) Mover uma encomenda da lista de pendentes para a fila de expedição;
- m) Enviar encomendas da fila para o histórico;
- n) Listar encomendas do histórico por intervalo de Ids;
- o) Listar encomendas do histórico por intervalo de datas;
- p) Listar encomendas do histórico por ID, nome ou data;

5. Desenvolvimento

❖ Estrutura de dados:

- i) Armazenar os dados das encomendas:

```
//Estrutura da encomenda
typedef struct Encomenda{
    int ID;
    char nome[100];
    char descricao[100];
    char dataDeCriacao[9]; //AAAAMMDD
    struct Encomenda *next;
}Encomenda;
```

- ii) Declarar a lista ligada das encomendas pendentes:

```
//estrutura das encomendas pendentes (lista ligada)
typedef struct {
    Encomenda *inicio;
}EncomendaPendente;
```

- iii) Gerir a fila de encomendas em expedição:

```
//estrutura das encomendas em expedicao (filas)
typedef struct {
    Encomenda *inicio, *fim;
}EncomendaExpedida;
```

- iv) Gerir a pilha do histórico de encomendas enviadas;

```
//estrutura do historico (pilhas)
typedef struct {
    Encomenda *top;
}Historico;
```

❖ Funções:

a) Validar datas no formato AAAAMMDD:

```
// Função para validar a data no formato AAAAMMDD
int validaData(const char *data) {
    int i;

    // Verifica se a data tem exatamente 8 caracteres
    if (strlen(data) != 8) {
        return 0; // Data inválida
    }

    // Verifica se cada caractere é um número (entre '0' e '9')
    for (i = 0; i < 8; i++) {
        if (data[i] < '0' || data[i] > '9') {
            return 0; // Data inválida
        }
    }

    // Extrai o mês (data[4] e data[5]) e converte para inteiro
    char mesStr[3] = { data[4], data[5], '\0' };
    int mes = atoi(mesStr); // atoi - converter para inteiro

    // Extrai o dia (data[6] e data[7]) e converte para inteiro
    char diaStr[3] = { data[6], data[7], '\0' };
    int dia = atoi(diaStr); // atoi - converter para inteiro

    // Verifica se o mês está entre 1 e 12
    if (mes < 1 || mes > 12) {
        return 0; // Mês inválido
    }

    // Verifica se o dia está entre 1 e 31
    if (dia < 1 || dia > 31) {
        return 0; // Dia inválido
    }

    // Verifica se os meses têm 30 dias
    if ((mes == 4 || mes == 6 || mes == 9 || mes == 11) && dia > 30) {
        return 0; // inválido
    }

    // Verifica se fevereiro tem 28 ou 29 dias
    if (mes == 2) {
        // Verifica se é um ano bissexto
        int ano = atoi(data); // Extrai o ano
        if ((ano % 4 == 0 && ano % 100 != 0) || (ano % 400 == 0)) {
            // Ano bissexto
            if (dia > 29) {
                return 0; // inválido
            }
        } else {
            // Ano não bissexto
            if (dia > 28) {
                return 0; // inválido
            }
        }
    }

    return 1; // Data válida
}
```

b) Verificar se já existe um ID em qualquer estrutura:

```
// Função para verificar se um ID já existe nas listas
int idExistente(const void *estrutura, int id, int tipo) {
    switch (tipo) {
        case 1: // Tipo 1: Lista de Encomendas
            const Encomenda *lista = (const Encomenda *)estrutura;
            while (lista != NULL) {
                if (lista->id == id) {
                    return 1; // ID já existe
                }
                lista = lista->next;
            }
            break;

        case 2: // Tipo 2: Lista de Encomendas Pendentes
            const EncomendaPendente *pendentes = (const EncomendaPendente *)estrutura;
            if (pendentes != NULL) {
                return idExistente(pendentes->inicio, id, HB01); // Reutiliza a verificação de lista
            }
            break;

        case 3: // Tipo 3: Lista de Encomendas Expedidas
            const EncomendaExpedida *expedidas = (const EncomendaExpedida *)estrutura;
            if (expedidas != NULL) {
                return idExistente(expedidas->inicio, id, HB01); // Reutiliza a verificação de lista
            }
            break;

        case 4: // Tipo 4: Pilha de Histórico
            const Historico *historico = (const Historico *)estrutura;
            if (historico != NULL) {
                return idExistente(historico->top, id, HB01); // Reutiliza a verificação de lista
            }
            break;

        default:
            break;
    }
    return 0; // ID não existe
}
```

c) Separar os campos de uma linha lida do ficheiro:

```
int separarElementos(char *linha, int *id, char *nome, char *descricao, char *dataCriacao) {
    char *token;

    // Copiar a linha para não destruir a original
    char copiaLinha[100];
    strcpy(copiaLinha, linha);

    token = strtok(copiaLinha, " ");
    if (!token || sscanf(sourceToken, "%d", id) != 1) return 0;

    token = strtok(NULL, " ");
    if (!token || strlen(token) == 0) return 0;
    strcpy(nome, token);

    token = strtok(NULL, " ");
    if (!token || strlen(token) == 0) return 0;
    strcpy(descricao, token);

    token = strtok(NULL, " ");
    if (!token || strlen(token) != 0) return 0;
    strcpy(dataCriacao, token);

    return 1; // Tudo certo
}
```

d) Ler o ficheiro de encomendas pendentes e carregar para a lista ligada:

```
// Função de transferência que lê o ficheiro para a lista ligada (Encomendas Pendentes)
void transferirPendentes(Encomenda **aplista) {
    char linha[100], nome[100], descricao[100], dataCriacao[20];
    int id;

    FILE *fPendentes = fopen(FNAME, "EncomendasPendentes.txt", "r");
    if (!fPendentes) {
        printf(FNAME, "Erro ao abrir EncomendasPendentes.txt");
        return;
    }

    // Ler cada linha de encomenda
    while (fgets(linha, MAXCOUNT, fPendentes)) {
        separarElementos(linha, &id, nome, descricao, dataCriacao);

        if (!validaData(dataCriacao)) {
            printf(FNAME, "Data inválida na encomenda %d id no ficheiro EncomendasPendentes.txt.\n", id);
            erros++;
        }

        // Verifica se o ID já existe e incrementa se necessário
        while (idExistente(aplista, id, HB02)) {
            id++;
        }

        Encomenda *nova = (Encomenda *)malloc(sizeof(Encomenda));
        if (!nova) {
            printf(FNAME, "Erro ao alocar memória.\n");
            return;
        }

        nova->id = id;
        strcpy(nova->nome, nome);
        strcpy(nova->descricao, descricao);
        strcpy(nova->dataCriacao, dataCriacao);
        nova->next = *aplista;
        *aplista = nova;

        if (id >= ultimoID) {
            ultimoID = id + 1;
        }
    }

    fclose(fPendentes);
}
```

f) Ler o ficheiro do histórico e carregar para a pilha:

```
//funcao de transferir que tem no ficheiro para a pilha (historico)
void transferirHistorico(Historico *stack) {
    char linha[100], nome[100], descricao[100], dataCriacao[20];
    int id;

    FILE *fHistorico = fopen("Fichameu:Historico.txt", "r");
    if (!fHistorico) {
        printf("Formato: Erro ao abrir Historico.txt");
        return;
    }

    while (fgets(linha, MAXCOUNT, sizeof(linha), fHistorico)) {
        separarElementos(linha, &id, nome, descricao, dataCriacao);

        //verifica a data
        if (!validaData(dataCriacao)) {
            printf("Formato: Data inválida na encomenda ID %d no ficheiro Historico.txt.\n", id);
            erros++;
        }

        // Verifica se o ID já existe e incrementa se necessário
        while (idExisteGera(stack, id, &id++)) {
            id++;
        }

        Encomenda *nova = (Encomenda *)malloc(sizeof(Encomenda));
        if (nova == NULL) {
            printf("Formato: Erro ao alocar memoria para encomenda.\n");
            return;
        }

        nova->ID = id;
        strcpy(nova->nome, nome);
        strcpy(nova->descricao, descricao);
        strcpy(nova->dataDeCriacao, dataCriacao);

        // inserir no topo da pilha
        nova->next = stack->top;
        stack->top = nova;

        if (id == ultimoID) {
            ultimoID = id + 1;
        }
    }

    fclose(fHistorico);
}
```

h) Inserir uma nova encomenda na lista de pendentes e atualizar o ficheiro:

```
// Funcao para capturar informacoes da encomenda
void capturaEntrada(char *prompt, char *buffer, int tamanho) {
    printf(format: "%s", prompt); // pergunta para o cliente
    fgets(buffer, tamanho, stdin); // guarda a resposta
}
```

```
//insere encomendas pendentes
void insereEncomendaPendente(Encomenda *apLista) {
    char nome[100], descricao[100], dataCriacao[0];
    fflush(stdin);

    capturaEntrada(prompt: "Introduza o nome do cliente: ", buffer: nome, tamanho: sizeof(nome));
    capturaEntrada(prompt: "Introduza a descricao da encomenda: ", buffer: descricao, tamanho: sizeof(descricao));

    do {
        capturaEntrada(prompt: "Introduza a data de criacao (AAAA-MM-DD): ", buffer: dataCriacao, tamanho: sizeof(dataCriacao));
        if (!validaData(dataCriacao)) {
            printf(format: "Formato de data invalido. use AAAA-MM-DD.\n");
        }
    } while (!validaData(dataCriacao));

    nome[strlen(Str: nome, Control: "\n")] = '\0';
    descricao[strlen(Str: descricao, Control: "\n")] = '\0';

    Encomenda *novaEncomenda = criaEncomenda(nome, descricao, dataCriacao);

    if (novaEncomenda == NULL) {
        printf(format: "Erro ao alocar memoria para a nova encomenda.\n");
        return;
    }

    novaEncomenda->next = *apLista; // Insere no inicio da lista
    *apLista = novaEncomenda;

    printf(format: "Encomenda com ID %d inserida com sucesso.\n", novaEncomenda->ID);

    //colocar as informacoes da encomenda pendente no ficheiro (EncomendasPendentes.txt)
    FILE *fPendentes = fopen( Fname: "EncomendasPendentes.txt", Mode: "w");
    if (!fPendentes) {
        printf(format: "Erro ao abrir o ficheiro EncomendasPendentes.txt.\n");
        return;
    }
}
```

```
//listar as encomendas pendentes
void listarEncomendasPendentes(Encomenda *head) {
    if (head == NULL) {
        printf(format: "Nao existem encomendas pendentes.\n");
        return;
    }

    printf(format: "\n          --- Encomendas Pendentes ---\n");

    Encomenda *atual = head;
    while (atual != NULL) {
        printf(format: "ID %10d Nome %30s Descricao %30s Data %10s\n", atual->ID, atual->nome, atual->descricao, atual->dataDeCriacao);
        atual = atual->next;
    }
    printf(format: "\n");
}
```

j) Listar todas as encomendas na fila de expedição:

```
//Listar as encomendas na fila de expedição
void front(const EncomendaExpedida *queue) {
    if (queue->inicio == NULL){
        printf(format: "Fila vazia.\n");
    }else{
        printf(format: "\n          --- Encomendas Expedidas ---\n");

        Encomenda *current = queue->inicio;
        while (current != NULL) {
            printf(format: "ID-%10d Nome-%20s Descrição-%30s Data-%10s\n", current->ID, current->nome, current->descricao, current->dataDeCriacao);
            current = current->next;
        }
        printf(format: "\n");
    }
}
```

k) Listar encomendas do histórico por nome do cliente, com filtros adicionais:

```
void nomeHistorico(const Historico* stack) {
    int opcao, encontrarNome=0, encontrou = 0;
    int idInicio = 0, idFim = 9999;
    char dataInicio[9] = "20000101", dataFim[9] = "99991231", buffer[10];
    char nome[100];

    Encomenda* atual = stack->top;

    do {
        getchar(); // Limpar o buffer
        printf(format: "\n--- Pesquisa por Nome ---\n");
        printf(format: "1 - Intervalo de IDs\n");
        printf(format: "2 - Intervalo de Datas\n");
        printf(format: "3 - Pesquisa simplesmente pelo nome\n");
        printf(format: "0 - Voltar atras\n");
        printf(format: "Escolha uma opcao: ");
        scanf(format: "%d", &opcao);

        fflush(stdin);

        if (opcao!=0) {
            // Validação para garantir que o nome existe na pilha
            do {
                printf(format: "Introduza o nome do cliente (ou parte do nome): ");
                fgets(nome, MaxCount:sizeof(nome), stdin);
                nome[strcspn(nome, "\n")] = '\0'; // remover newline

                encontrarNome=0;
                Encomenda* nova = stack->top;

                while (nova != NULL) {
                    if (strstr(nome, SUBSTR(nova)) != NULL) {
                        encontrarNome = 1;
                        break;
                    }
                    nova = nova->next;
                }

                if (!encontrarNome) {
                    printf(format: "Nenhuma encomenda encontrada com esse nome. Tente novamente.\n");
                }
            } while (!encontrarNome);

            switch (opcao) {
                case 1:
                    fflush(stdin);

                    do {
                        char buffer[10];
                        printf(format: "Introduza o ID de inicio: ");
                        fgets(buffer, MaxCount:sizeof(buffer), stdin);
                        if (strlen(buffer) > 1) { // Se o utilizador não pressionou apenas Enter
                            idInicio = atoi(buffer);
                        }

                        printf(format: "Introduza o ID de fim: ");
                        fgets(buffer, MaxCount:sizeof(buffer), stdin);
                        if (strlen(buffer) > 1) {
                            idFim = atoi(buffer);
                        }

                        if (idInicio > idFim || idInicio < 0 || idFim < 0) {
                            printf(format: "IDs inválidos. Tente novamente.\n");
                        }
                    } while (idInicio > idFim || idInicio < 0 || idFim < 0);

                    printf(format: "\n          --- Encomendas de %s entre %s e %s ---\n", nome, idInicio, idFim);
                    while (atual != NULL) {
                        if (strstr(atual->nome, SUBSTR(nome)) != NULL && strcmp(atual->ID, idInicio) >= 0 && strcmp(atual->ID, idFim) <= 0) {
                            printf(format: "ID-%10d Nome-%20s Descrição-%30s Data-%10s\n", atual->ID, atual->nome, atual->descricao, atual->dataDeCriacao);
                            encontrou = 1;
                            fflush(stdin);
                        }
                        atual = atual->next;
                    }
                    if (!encontrou) {
                        printf(format: "Nenhuma encomenda encontrada com esse nome nesse intervalo de IDs.\n");
                    }
                    break;
                case 2:
                    fflush(stdin);

                    // Obter e validar data de início
                    do {
                        printf(format: "Introduza a data de início (AAAA-MM-DD): ");
                        fgets(buffer, MaxCount:sizeof(buffer), stdin);
                        if (strlen(buffer) > 1) {
                            strcpy(dataInicio, buffer);
                            dataInicio[strcspn(dataInicio, "\n")] = '\0'; // Remove newline
                        }
                        if (!validaData(dataInicio)) {
                            printf(format: "Data inválida. Tente novamente.\n");
                        } else if (strcmp(dataInicio, dataFim) > 0) {
                            printf(format: "A data de fim deve ser posterior ou igual a data de início.\n");
                        }
                    } while (!validaData(dataFim) || strcmp(dataInicio, dataFim) > 0);
                }
            }
        }
    } while (opcao != 0);
}
```

k) Continuação listar encomendas do histórico por nome do cliente, com filtros adicionais:

```
Encomenda* atual2 = stack->top;
printf(format: "\n          --- Encomendas de %s entre %s e %s ---\n", nome, dataInicio, dataFim);
while (atual2 != NULL) {
    if (strstr(atual2->nome, SUBSTR(nome)) != NULL && strcmp(atual2->dataDeCriacao, dataInicio) >= 0 && strcmp(atual2->dataDeCriacao, dataFim) <= 0) {
        printf(format: "ID-%10d Nome-%20s Descrição-%30s Data-%10s\n", atual2->ID, atual2->nome, atual2->descricao, atual2->dataDeCriacao);
        encontrou = 1;
    }
    atual2 = atual2->next;
}
if (!encontrou) {
    printf(format: "Nenhuma encomenda encontrada com esse nome nesse intervalo de datas.\n");
}
break;
case 3: encontrou = 0;
Encomenda* atual3 = stack->top;
printf(format: "\n          --- Encomendas com o nome '%s' ---\n", nome);
while (atual3 != NULL) {
    if (strstr(atual3->nome, SUBSTR(nome)) != NULL) {
        printf(format: "ID-%10d Nome-%20s Descrição-%30s Data-%10s\n", atual3->ID, atual3->nome, atual3->descricao, atual3->dataDeCriacao);
        encontrou = 1;
    }
    atual3 = atual3->next;
}
if (!encontrou) {
    printf(format: "Nenhuma encomenda encontrada para o nome '%s'.\n", nome);
}
break;
default: printf(format: "Opcao inválida. Tente novamente.\n");
}
return;
}
while (opcao != 0);
}
```

m) Enviar encomendas da fila para o histórico:

```

remover encomenda e expedidas (enviar para o cliente) e acrescentar encomenda ao historico
void push(EncomendaExpedida *queue, Historico *stack) {
    int r, contador=0;

    if (queue->inicio == NULL) {
        printf(format:"Nao ha encomendas expedidas.\n");
        return;
    }

    const Encomenda *fila = queue->inicio;
    while (fila != NULL) {
        fila=fila->next;
        contador++;
    }

    printf(format:"Quantas encomendas pretende enviar? ");
    do {
        scanf(format:"%d", &r);
        if (r < 0 || r > contador)
            printf(format:"Apenas %d encomendas em expedicao disponiveis para envio, introduza novamente ", contador);
    }while (r < 0 || r > contador);

    while (r > 0 && queue->inicio != NULL) {
        Encomenda *enviada = queue->inicio; // Encomenda a ser enviada
        queue->inicio = enviada->next; // Avançar a fila
        if (queue->inicio == NULL) {
            queue->fim = NULL; // Se a fila ficar vazia
        }

        // Inserir no topo de pilha (histórico)
        enviada->next = stack->top;
        stack->top = enviada;

        printf(format:"Encomenda com ID %d enviada para o cliente e movida para o historico.\n", enviada->ID);

        r--; // Reduzir o numero de encomendas restantes a enviar
    }

    if (r > 0) {
        printf(format:"Apenas existiam %d encomendas para enviar.\n", r);
    }

    // Atualizar o ficheiro de encomendas expedidas
    FILE *fExpedidas = fopen(FileName:"EncomendasExpedidas.txt", (Mode:"w"));
    if (!fExpedidas) {
        printf(format:"Erro ao abrir EncomendasExpedidas.txt");
        return;
    }

    if (queue->inicio != NULL) {
        Encomenda *temp = queue->inicio;
        while (temp != NULL) {
            fprintf(fExpedidas, format:"%d;%s;%s;\n", temp->ID, temp->nome, temp->descricao, temp->dataDeCriacao);
            temp = temp->next;
        }
    }

    fclose(fExpedidas);

    // Atualizar o ficheiro do histórico
    FILE *fHistorico = fopen(FileName:"Historico.txt", (Mode:"a"));
    if (!fHistorico) {
        printf(format:"Erro ao abrir Historico.txt");
        return;
    }

    Encomenda *tempHist = stack->top;
    while (tempHist != NULL) {
        fprintf(fHistorico, format:"%d;%s;%s;\n", tempHist->ID, tempHist->nome, tempHist->descricao, tempHist->dataDeCriacao);
        tempHist = tempHist->next;
    }

    fclose(fHistorico);
}

```

```
#include <stdio.h> #define HISTORIA_Stack stack<Historia*> typedef struct { int idInicio = 0, idFim = 9999, encotrou = 0; char buffer[60]; } Historias; void print(Historias *stack) { do { printf(formato "Introduza o ID de inicio: "); fgets(buffer, MAXCOUNT, sizeof(buffer), stdin); if ((strlen(buffer) > 1) || // Se o utilizador não pressionou apenas Enter idInicio == atoi(buffer)); } while (!idInicio); printf(formato "Introduza o ID de fim: "); fgets(buffer, MAXCOUNT, sizeof(buffer), stdin); if ((strlen(buffer) > 1) || idFin == atoi(buffer)); } while (!idFin); if (idInicio > idFin || idInicio <= 0 || idFin <= 0) { printf(formato "IDs invalidos. Tente novamente.\n"); return; } while (idInicio > idFin || idInicio <= 0 || idFin <= 0); // Encerrar e lista e imprimir enconturas dentro do intervalo Enconturas atual = stack->top; ... Enconturas no intervalo de IDs N1 -> N2, idInicio, idFin); printf(formato "\n\n"); while (atual != NULL) { if (atual->id > idInicio && atual->id <= idFin) { printf(formato "ID %d00 base %d000 Encontra-se No Base %d00A's, atual-ID1, atual-base, atual-IdDescricao, atual-VetoridadeLocal); encotrou ++ ; } atual = atual->nxt; } if (!encotrou) { printf(formato "Nenhuma encontura encontrada no intervalo N1 -> N2, idInicio, idFin);\n"); }
```

o) Listar encomendas do histórico por intervalo de datas:

```
void dataHistorico(const Historico stack) {
    char dataInicio[0] = "20000101", dataFim[0] = "99991231", buffer[10];
    int encontrow = 0;

    fflush(stdin);

    // Opção e validar data de início
    do {
        printf(formato"Introduza a data de início (AAAA-MM-DD): ");
        fgets(buffer, MAX_COUNT*sizeof(buffer), stdin);
        if (strlen(buffer) > 1) {
            strcpy(dataInicio, buffer);
            dataInicio[strlen(sizeof(dataInicio), Control "\n")] = '\0'; // Remove newline
        }

        if (invalidaData(dataInicio)) {
            printf(formato"Data inválida. Tente novamente.\n");
        }
    } while (invalidaData(dataInicio));

    // Opção e validar data de fim
    do {
        printf(formato"Introduza a data de fim (AAAA-MM-DD): ");
        fgets(buffer, MAX_COUNT*sizeof(buffer), stdin);
        if (strlen(buffer) > 1) {
            strcpy(dataFim, buffer);
            dataFim[strlen(sizeof(dataFim), Control "\n")] = '\0'; // Remove newline
        }

        if (invalidaData(dataFim)) {
            printf(formato"Data inválida. Tente novamente.\n");
        } else if (strcmp(dataInicio, dataFim) > 0) {
            printf(formato"A data de fim deve ser posterior ou igual a data de início.\n");
        }
    } while (invalidaData(dataFim) || strcmp(dataInicio, dataFim) > 0);

    // Percorrer lista e imprimir encerradas no intervalo
    encerradasAtual = stack->top;
    printf(formato"\n\t\t\t\t\tEncerradas entre as datas %s e %s\n\n", dataInicio, dataFim);

    while (atual != NULL) {
        if (strcmp(atual->dataEntradas, dataInicio) < 0 && strcmp(atual->dataEntradas, dataFim) <= 0) {
            printf(formato"%10s-%10s Nome %s-20s Encerrado %s-10s Data %s-10s\n", atual->id, atual->nome, atual->descricao, atual->dataEntradas, atual->dataEncerradas);
            encontrow++;
        }
        atual = atual->next;
    }

    if (encontrow) {
        printf(formato"Encontrei %d encerradas no intervalo de datas %s\n", encontrow, dataInicio);
    }
}
```

p) Listar encomendas do histórico por nome do cliente, com filtros adicionais:

```
//listar encomendas expedidas
void listarHistorico(const Historico* stack) {
    int r;

    if (stack->top == NULL) {
        printf(format:"Historico vazio.\n");
        return;
    }

    do {
        printf(format:"\n--- Escolha uma opcao para listar o historico ---\n");
        printf(format:"1-Intervalo de IDS\n");
        printf(format:"2-Intervalo de datas\n");
        printf(format:"3-Pesquisar por nome\n");
        printf(format:"4-Mostrar tudo\n");
        printf(format:"0-Voltar atras\n");
        scanf(format:"%d", &r);

        switch (r) {
            case 1: idHistorico(stack);
                    break;

            case 2: datasHistorico(stack);
                    break;

            case 3: nomeHistorico(stack);
                    break;

            case 4: printf(format:"\n
                                --- Historico de Encomendas ---\n");
                    while (Atual != NULL) {
                        printf(format:"%10s-%10s Nome-%20s Descricao-%30s Data-%10s\n", atual->nome, atual->descricao, atual->dataDeCriacao);
                        atual = atual->next;
                    }
                    printf(format:"\n");
                    break;

            case 0: printf(format:"Origem\n");
                    break;

            default: printf(format:"Opcoes Invalidas. Introduza de 1 a 4");
                    break;
        }
    } while (r!=0);
}
```


6. Anexos

6.1. Teste de funções:

```
int main() {
    int res;
    setlocale( Category: LC_ALL,  Locale: "pt-PT.UTF-8");

    //listas ligadas
    Encomenda *head = NULL;

    //filas
    EncomendaExpedida EE;
    //inicializar a fila
    EE.inicio = NULL;
    EE.fim = NULL;

    //pilhas
    Historico stack;
    stack.top = NULL;

    //transferir os ficheiros para as respetivas areas
    transferirPendentes(&head);
    transferirExpedidas(&EE);
    transferirHistorico(&stack);

    if (erros > 0) {
        printf(format:"\nForam encontrados %d erros ao carregar os ficheiros. Corrija-os e tente novamente.\n", erros);
        exit( Code:1);
    }

    do {
        printf(format:"--- Escolha uma opcao ---\n");
        printf(format:"1-Introduzir uma encomenda\n");
        printf(format:"2-Listar encomendas pendentes\n");
        printf(format:"3-Expedir uma encomenda\n");
        printf(format:"4-Listar encomendas em expedicao\n");
        printf(format:"5-Enviar encomenda\n");
        printf(format:"6-Listar as encomendas enviadas\n");
        printf(format:"0-Sair\n");
        scanf(format:"%d", &res);

        switch (res) {
            case 1: insereEncomendaPendente(&head);
                    break;
            case 2: listarEncomendasPendentes(head);
                    break;
            case 3: enqueue(&head, &EE);
                    break;
            case 4: front(&EE);
                    break;
            case 5: push(&EE, &stack);
                    break;
            case 6: listarHistorico(&stack);
                    break;
            case 0: printf(format:"Obrigada");
                    break;
            default: printf(format:"Erro, introduza um numero entre 0 e 6.\n");
        }
    }while (res!=0);
    return 0;
}
```

Dados no ficheiro de texto:

```
2;Carla Pinto; Pasteis de Óleo;20250426
70;Fábio Vieira;Tesoura;20250813
69;Beatriz Duarte;Lápis;20250611
68;Filipe Henriques;Fita cola;20250320
67;Cláudio Neves;Marcador;20250108
66;Carla Barros; Bloco de notas;20251202
65;Renato Pinto; Caneta de gel;20251119
64;Paula Leal;Afiador;20251001
63;Joana Cardoso;Separadores;20250912
62;Bruno Tavares;Agrafador;20250828
61;Miguel Rocha;Cartolina;20250705
60;Inês Faria;Borracha;20250616
```